

Python高级应用

Python的模块和库



知识目标

1. 理解Python的模块和包的概念
2. 了解常用的可视化库
3. 理解爬虫的基本原理、Web的三件套



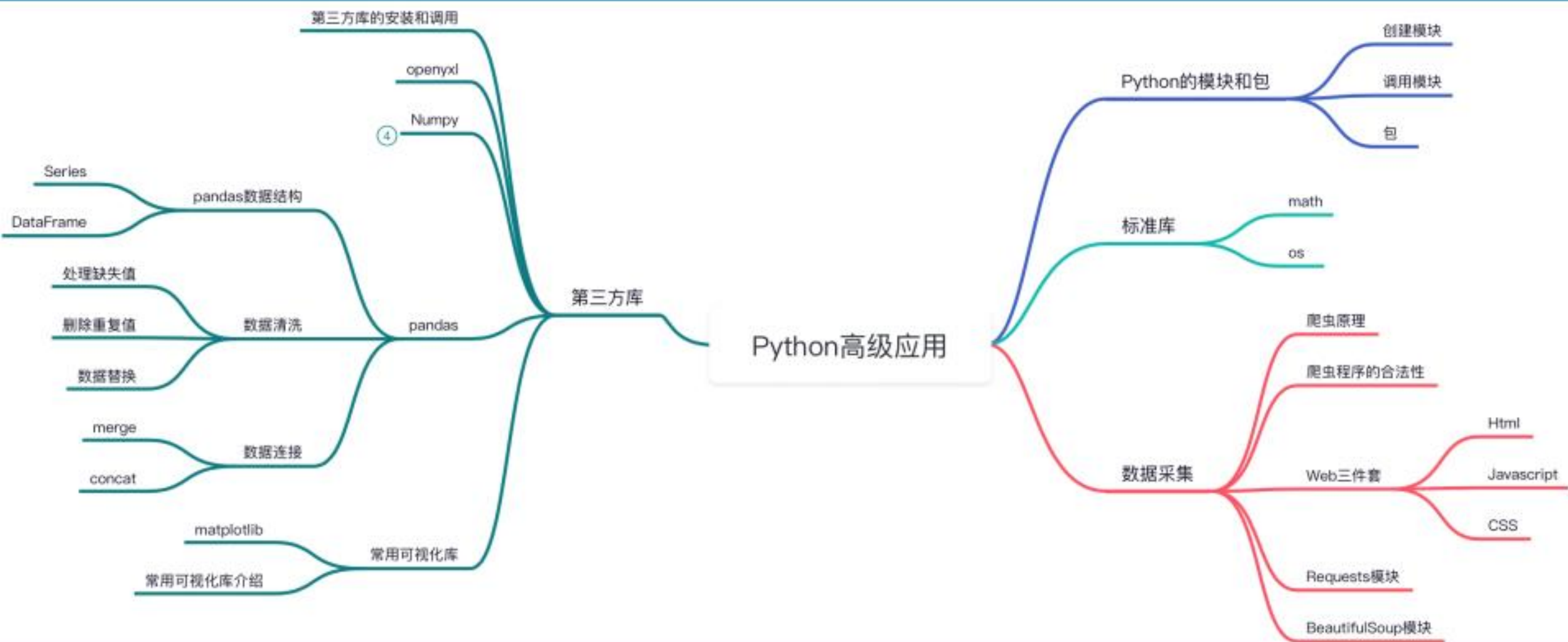
能力目标

1. 掌握Python模块的安装和调用
2. 掌握os、math标准库的初步应用
3. 掌握第三方库的安装与调用
4. 掌握openpyxl、numpy、pandas数据处理库的初步应用
5. 掌握matplotlib库的初步应用
6. 掌握Requests、模块的初步应用



素质目标

1. 具备代码的结构思维能力
2. 树立持续学习的理念



1 案例背景

2 知识准备

3 任务要求

4 任务解析

“如果我比别人看的更远，那是因为我站在巨人的肩膀上”，Python之所以强大，很大程度得益于其开源活跃的生态系统。近年来，Python提高了对类库的支持，除了标准库，Python语言还有超过12万个第三方库，可以帮助解决文档的管理、数据分析与数据科学等各类问题。



1 案例背景

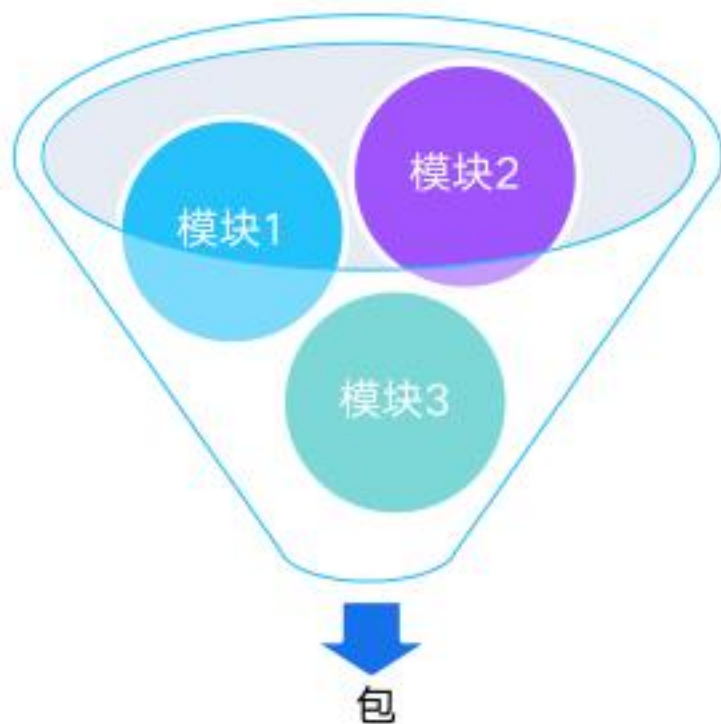
2 知识准备

3 任务要求

4 任务解析

在Python程序的开发过程中，为了代码维护和重复使用的方便，将一些可以实现相对独立功能的代码存储为一个后缀名为.py的文件，每个.py文件就是一个模块（Module）。

包就是多个模块的集合。当项目较大，模块较多时，我们就可以把模块放在包中，便于管理。



创建模块，只需将所需代码保存在文件扩展名为.py的文件中。

创建模块

示例：打开文本文件，编写一个自定义函数，计算一年内多次计息的实际利率，并保存为.py文件

#计算一年内多次计息的实际利率，r为名义利率，m为一年内的计息次数

```
def actual(r,m):  
    return (1+r/m)**m - 1
```

创建模块，只需将所需代码保存在文件扩展名为.py的文件中。

调用模块

模块的调用通过import语句实现，如果模块名称较长，可以通过as将其命名为一个简称，此时调用其中的自定义函数时，需要在函数前添加模块名：

```
import actualinterest as ai  
ai.actual(0.08,12)
```

也可以通过from……import语句只调用模块中的某个对象，此时调用其中的自定义函数时，不需在函数前添加模块名：

```
from actualinterest import actual  
actual(0.08,12)
```

按惯例，所有import语句都放在模块（或脚本）开头。

练习2.1-1：在上述.py文件中添加一个自定义函数，根据给定的名义利率和通货膨胀率计算实际利率，并在python环境中调用该函数，以9%的名义利率和5%的通货膨胀率带入进行测算实际利率。



标准库是Python官方内置的一些模块，我们安装Python的时候，就已经把这些库集成到我们的电脑本地，可以直接使用import导入使用。在财务管理应用中常用的标准库包括os、datetime、math等。



os就是“operating system”的缩写，顾名思义，os模块提供的就是各种Python 程序与操作系统进行交互的接口，包含几百个函数。通过使用os模块，可以进行常用路径操作、进程管理和设置环境参数等，在财务管理中常用的应用如下：

- 模块导入：import os
- 获得当前目录：os.getcwd()
- 列出当前目录下的文件：os.listdir()
- 切换目录：os.chdir()

在括号中填写路径即可切换到相应目录，“..”可切换到上一级目录。

- 遍历目录：os.walk()
- 拼接路径：os.path.join()

遍历目录

代码: `list(os.walk("."))`

该语句与`os.listdir`的区别是`os.walk`将当前目录下的所有子目录及其中的内容, 都会遍历到; 而`os.listdir`只会遍历当前目录中所包含的内容。`os.walk()`缺省后面的参数量返回的是一个三元组(`root,dirs,files`), 其中`root`所指的是当前正在遍历的这个文件夹的本身的地址; `dirs`是一个list, 内容是该文件夹中所有的目录的名字(不包括子目录), `files`也是一个list, 内容是该文件夹中所有的文件(不包括子目录)。

示例:

```
root_dir = os.getcwd()
for files in os.walk(root_dir):
    print(files)
```

拼接路径

代码: `os.path.join()`

该语句将参数中的路径进行拼接, 例如`os.path.join("xx", "yy", "zz")`可输出路径为: `xx/yy/zz`。在获取文件时, 由于文件路径与文件名是分开的, 因此对文件进行处理时常利用该语句对文件路径和文件名进行拼接。以下拼接的示例也显示了`os.listdir`与`os.walk`的区别:

示例1:

```
dir1 = os.getcwd()
for file in os.listdir():
    print(os.path.join(dir1, file))
```

示例2:

```
root_dir = os.getcwd()
for files in os.walk(root_dir):
    root, dirs, file = files
    for f in file:
        print(os.path.join(root, f))
```

练习2.1-2：导入os标准库，输出当前目录下的所有文件的“路径+文件名”组合。



在财务管理的应用中，会应用到大量的公式，而Python中的math库提供了非常实用的计算函数，以下是常用函数示例：

函数	返回值
ceil(x)	大于或等于x的整数
exp(x)	e的x次方
factorial(n)	计算n的阶乘(n!)
log(x)	以e为底的x的对数
log(x,b)	以b为底的x的对数
pow(x,y)	x的y次方
sqrt(x)	x的平方根

math

导入math模块，根据经济定货批量基本模型自定义经济批量定货的函数，其中Q表示经济定货批量，c为单次订货成本，r为总需求量，h为单位产品的库存成本：

```
import math  
def Q(c,r,h):  
    return math.sqrt(2*c*r/h)
```

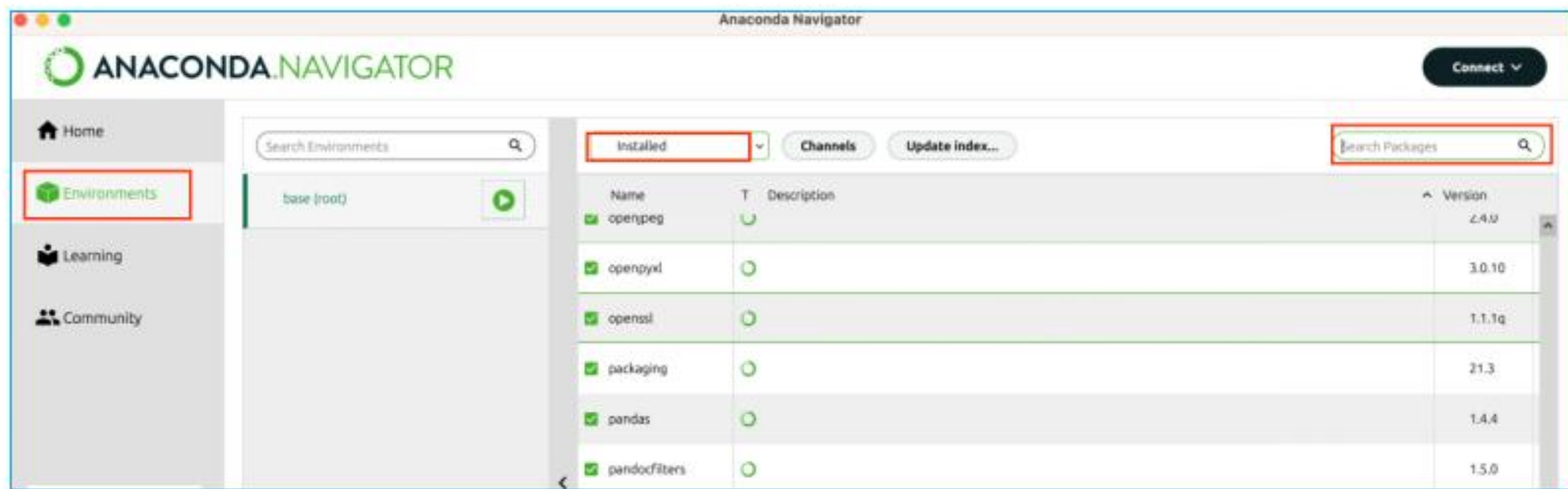
```
Q(5000,10000,50)
```

练习2.1-3：导入math模块，计算下列公式的数值：

$$\frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad (a=2, b=4, c=0.2)$$



第三方库由Python社区提供，跟标准库不同的是，要使用第三方模块，需要使用pip安装。如果我们通过安装Anaconda来使用Python，该程序已自动安装常用的第三方库，可以在下图的页面进行查看或搜索：



安装

打开命令窗口，输入“pip install 库名”即可。

示例：安装一个pdf2docx的第三方库，并导入其中的Converter函数将PDF文件转换为word文档。

打开命令窗口（mac系统打开终端输），输入“pip install pdf2docx”安装pdf2docx库：

```
1 | pip install pdf2docx
```

调用

在Python环境中通过import调用：

```
from pdf2docx import Converter  
pdf_file = "Hello,Python.pdf"  
docx_file = "Hello,Python.docx"  
cv = Converter(pdf_file)  
cv.convert(docx_file, start=0, end=None)  
cv.close()
```

openpyxl库是一个比较综合的工具，可以处理Excel2007及以上版本产生的xlsx文件，能够同时读取和修改Excel文档。

基本用法

创建工作簿和和工作表

示例：

```
import openpyxl
wb = openpyxl.Workbook()          #创建工作簿
ws = wb.create_sheet("Mysheet")  #创建工作表
wb.save("77.xlsx")
```

读取Excel文件

示例：

```
wb = openpyxl.load_workbook("77.xlsx")  #打开77.xlsx文件
ws = wb["Mysheet"]                      #使用工作表名获取该文件中的工作表
```

基本用法

Excel单元格操作

示例:

<code>ws["A2"] = 2</code>	<code>#为单元格赋值</code>
<code>ws["A3"] = 3</code>	<code>#为单元格赋值</code>
<code>ws["A1"] = "=SUM(A2:A3)"</code>	<code>#编辑单元格公式</code>
<code>ws.merge_cells("B2:B3")</code>	<code>#合并单元格</code>
<code>wb.save("77.xlsx")</code>	
<code>ws["A2"].value</code>	<code>#获取单元格的值</code>

遍历数据表中的数据

示例:

```
for row in ws.values:
    for value in row:
        print(value)
```

练习2.1-4：导入openpyxl，读取提供的Excel文件“营业收入.xlsx”，输出“revenue”表格中所有的值。



Numpy是Numerical Python的简写，是目前Python数值计算中最为重要的基础包，它提供多种数据结构、算法以及大部分涉及Python数值计算所需的接口，同时也可作为算法与库之间作为数据传递的容器。对于数值计算，Numpy能够比Python内建数据结构更为高效的存储和操作数据。Numpy最为核心的数据类型是ndarray，使用ndarray可以处理一维、二维和多维数组。



创建ndarray对象（数组）

创建数组最简单的方法是使用array函数，array函数接收任意的序列型对象（例如：字符串、列表、元组……），生成一个新的包含传递数据的数组。

创建一维数组：

```
import numpy as np          #导入numpy，np为numpy的惯常简写
array1 = np.array([1,2,3,4,5]) #创建一维数组
array1

array2 = np.arange(10)      #arange是Python内建函数的数组版
array2
```

创建ndarray对象（数组）

创建二维数据：

```
data1 = [[1,2],[3,4],[5,6]]
```

```
array3 = np.array(data1)          #创建二维数据
```

```
array3
```

data1是一个包含列表的列表，因此生成了一个二维数组

创建ndarray对象（数组）

可以通过shape属性查看其形状

```
array3.shape
```

通过.reshape属性可以对数组的形状进行转换：

```
array2.reshape(2,5)
```

创建ndarray对象（数组）

除np.array之外，还有一些函数也可以新建数组。比如，zeros和ones分别可以创建指定长度或形状的全0或全1数组。要用这些方法创建多维数组，只需传入一个表示形状的元组即可：

```
array4 = np.zeros((3,6))
```

```
array4
```

```
array5 = np.ones((5,2))
```

```
array5
```

数组的运算

数组可以不使用for循环而进行批量运算，因此其数值运算的效率远高于Python方法，占用更少的内存。两个形状相同的数组运算对数组中的每个元素逐个对应计算：

```
array6 = np.array([[1,2,3],[4,5,6]])
```

```
array6 * array6                                #数组的相乘
```

```
array6 - array6                                #数组的相减
```

数组的运算

数组与标量进行运算，标量会与数组中的每一个元素进行计算：

```
array6 * 3
```

两个形状相同的数组进行比较，会输出一个布尔值数组：

```
array7 = np.array([[3,2,1],[6,5,4]])
```

```
array6 > array7
```

数组的运算

可以运用数学函数对整个数组进行统计计算。

示例：基础数组统计方法

```
array11 = np.arange(10)
```

```
array11
```

```
array11.sum()           #求数组的元素和
```

```
array11.mean()          #求数组的元素平均值
```

```
array11.cumsum()         #数组的元素累积和
```

其他基础数组统计方法:

方法	说明
sum	对数组中全部或者某轴向的元素求和(零长度的数组的sum为0)
mean	算数平均数(零长度的数组的mean为NaN)
std, var	分别为标准差和方差,自由度可调(默认为n)
min, max	最大值和最小值
argmin, argmax	分别为最大和最小元素的索引
cumsum	所有元素的累计和
cumpord	所有元素的累计积

数组的运算

数组可以使用通用函数对其中元素进行计算或操作，其中一元函数接收一个标量数值并产生一个标量结果，二元函数接收两个数组进行计算并返回一个数组。

示例：一元函数的应用

```
array12 = np.arange(10)
```

```
array12
```

```
array13 = np.sqrt(array11)
```

#对数组元素进行平方根计算

```
array13
```

```
np rint(array13)
```

#对数组元素按四舍五入进行取整

一元通用函数：

函数	说明
abs / fabs	求绝对值的函数
sqrt	求平方根的函数，相当于 <code>array ** 0.5</code>
square	求平方的函数，相当于 <code>array ** 2</code>
exp	计算 e^x 的函数
log / log10 / log2	对数函数（e为底 / 10为底 / 2为底）
sign	符号函数（1 - 正数；0 - 零；-1 - 负数）
ceil / floor	上取整 / 下取整
isnan	返回布尔数组，NaN对应True，非NaN对应False
isfinite / isinf	判断数值是否为无穷大的函数
cos / cosh / sin	三角函数
sinh / tan / tanh	三角函数
arccos / arccosh / arcsin	反三角函数
arcsinh / arctan / arctanh	反三角函数
rint / round	四舍五入函数

数组的运算

示例：二元函数的应用

```
array14 = np.arange(10,50,3)
```

```
array14
```

```
array15 = np.arange(0,80,6)
```

```
array15
```

```
np.maximum(array14,array15)
```

#比较两个数组并选取对应元素中的最大值返回一个数组

二元通用函数：

函数	说明
add(x, y) / subtract(x, y)	加法函数 / 减法函数
multiply(x, y) / divide(x, y)	乘法函数 / 除法函数
floor_divide(x, y) / mod(x, y)	整除函数 / 求模函数
allclose(x, y)	检查数组x和y元素是否几乎相等
power(x, y)	数组x的元素 x_i 和数组y的元素 y_i ，计算 $x_i^{y_i}$
maximum(x, y) / fmax(x, y)	两两比较元素获取最大值 / 获取最大值（忽略NaN）
minimum(x, y) / fmin(x, y)	两两比较元素获取最小值 / 获取最小值（忽略NaN）
dot(x, y)	点积运算（数量积，通常记为 $\cdot$ ，用于欧几里得空间（Euclidean space））
inner(x, y)	内积运算（内积的含义要高于点积，点积相当于是内积在欧几里得空间的特例，而内积可以推广到赋范向量空间，只要它满足平行四边形法则即可）
cross(x, y)	叉积运算（向量积，通常记为 $\times$ ，运算结果是一个向量）
outer(x, y)	外积运算（张量积，通常记为 $\otimes$ ，运算结果通常是一个矩阵）
intersect1d(x, y)	计算x和y的交集，返回这些元素构成的有序数组
union1d(x, y)	计算x和y的并集，返回这些元素构成的有序数组
in1d(x, y)	返回由判断x 的元素是否在y中得到的布尔值构成的数组
setdiff1d(x, y)	计算x和y的差集，返回这些元素构成的数组
setxor1d(x, y)	计算x和y的对称差，返回这些元素构成的数组

切片和索引

一维数组的索引比较简单，和Python的列表类似：

示例：

```
array8 = np.arange(10)
```

```
array8[3]                #获取一维数组的第4个元素
```

```
array8[5:8]              #获取一维数组的第6-9个元素
```

切片和索引

可以将数值直接传递给数组的索引或切片而修改数组元素：

示例：

```
array8[5:8] = 10
```

```
array8
```

切片和索引

对于二维数组，每个索引值对应的是一个一维数组：

示例：

```
array9 = np.array([[1,2,3],[4,5,6],[7,8,9]])
```

```
array9[2]
```

如果需要获取其中的元素需要定义数组的坐标，其坐标如图所示：

对二维数组进行切片：

```
array9[1,2]          #获取第2行第3列的元素
```

```
array9[:2]           #获取前两行的元素
```

```
array9[:]            #获取整个数组
```

	0	1	2
0	0,0	0,1	0,2
1	1,0	1,1	1,2
2	2,0	2,1	2,2

练习2.1-5：创建一个8行4列的零值数值，然后通过for循环将第一行赋值为0，第二行赋值为1……以此类推。



数组的转置

数组具有特殊的T属性，可以将其行列进行转换：

示例：

```
array16 = np.arange(15).reshape(3,5)
```

```
array16
```

```
array16.T
```

1 案例背景

2 知识准备

3 任务要求

4 任务解析

附表2-1是招商银行2016-2021年各年的营业收入数据，要求通过openpyxl读取数据，利用Numpy将数据创建为一维数组，并求解各年收入的和、平均值与最大值。



1 案例背景

2 知识准备

3 任务要求

4 任务解析

根据任务要求，要用到哪些Python库与用法？



- ✓ 任务给出excel表格，因此需要利用学到的openpyxl，利用循环读取表格中的数据；
- ✓ 其次需要创建一维数据，因此需要使用Numpy库进行创建；
- ✓ 最后可以使用Numpy中的数组统计方法，.sum()、.mean()和max()。



```
import openpyxl as op
import numpy as np
wb = op.load_workbook("附表2-1.xlsx")
ws = wb["Sheet1"]
a = []
for row in ws["A1":"A5"]:
    for revenue in row:
        a.append(revenue.value)

b = np.array(a)
print("收入合计: ",b.sum())
print("平均收入: ",b.mean())
print("最大收入: ",b.max())
```



编写一个自定义函数，用两个一维数组实现根据收益率和概率计算预期收益率，并存储为.py文件，在Python环境调用，根据提供的附表2-2中的数据计算其预期收益率。



感謝
ขอบคุณ
terima kasih
ありがとう
Thanks
谢谢